

ML INTERVIEW QUESTIONS

1. Basics of Machine Learning

Q1. What is Machine Learning?

Machine Learning is a subset of Artificial Intelligence that enables systems to learn patterns from data and make decisions without being explicitly programmed. It uses statistical techniques to improve performance on a task through experience.

Q2. What are the types of Machine Learning?

Machine Learning is broadly classified into supervised learning, unsupervised learning, and reinforcement learning. Supervised learning uses labeled data, unsupervised learning works with unlabeled data to find patterns, and reinforcement learning involves learning through rewards and penalties.

Q3. What is the difference between AI, ML, and Deep Learning?

Artificial Intelligence is the broader concept of machines performing intelligent tasks, Machine Learning is a subset focused on learning from data, and Deep Learning is a subset of ML that uses neural networks with multiple layers to model complex patterns.

2. Supervised Learning

Q4. What is supervised learning?

Supervised learning is a type of ML where the model is trained on labeled data, meaning both input and output are known. The goal is to learn a mapping from input to output.

Q5. What is regression?

Regression is a supervised learning task where the output variable is continuous, such as predicting house prices.

Q6. What is classification?

Classification is a supervised learning task where the output variable is categorical, such as spam detection.

3. Common Algorithms

Q7. What is Linear Regression?

Linear Regression is an algorithm that models the relationship between input variables and a continuous output using a linear equation. It assumes a linear relationship between variables.

Q8. What is Logistic Regression?

Logistic Regression is used for classification problems and outputs probabilities using a sigmoid function.

Q9. What is Decision Tree?

A Decision Tree is a tree-like model where decisions are made based on feature values, splitting data recursively.

Q10. What is Random Forest?

Random Forest is an ensemble method that builds multiple decision trees and combines their outputs to improve accuracy and reduce overfitting.

4. Model Evaluation

Q11. What is accuracy?

Accuracy is the ratio of correctly predicted observations to total observations.

Q12. What is precision and recall?

Precision measures how many predicted positives are actually correct, while recall measures how many actual positives were correctly identified.

Q13. What is F1-score?

F1-score is the harmonic mean of precision and recall, providing a balance between them.

5. Overfitting & Underfitting

Q14. What is overfitting?

Overfitting occurs when a model learns noise in the training data and performs poorly on unseen data.

Q15. What is underfitting?

Underfitting occurs when a model is too simple to capture underlying patterns in the data.

6. Regularization

Q16. What is regularization?

Regularization is a technique used to prevent overfitting by adding a penalty term to the loss function.

Q17. What is L1 vs L2 regularization?

L1 regularization adds absolute values of coefficients and can shrink some to zero, while L2 adds squared values and reduces magnitude but not to zero.

7. Unsupervised Learning

Q18. What is clustering?

Clustering groups similar data points together without labeled data.

Q19. What is K-Means?

K-Means partitions data into K clusters by minimizing the distance between points and cluster centroids.

8. Dimensionality Reduction

Q20. What is PCA?

Principal Component Analysis (PCA) reduces dimensionality by transforming data into new variables that maximize variance.

9. Model Training Concepts

Q21. What is training and testing data?

Training data is used to train the model, while testing data evaluates its performance.

Q22. What is cross-validation?

Cross-validation splits data into multiple folds to ensure robust model evaluation.

10. Bias-Variance Tradeoff

Q23. What is bias?

Bias refers to error due to overly simplistic assumptions in the model.

Q24. What is variance?

Variance refers to error due to model sensitivity to small fluctuations in training data.

11. Optimization

Q25. What is gradient descent?

Gradient descent is an optimization algorithm used to minimize loss by updating parameters iteratively.

12. Loss Functions

Q26. What is loss function?

A loss function measures how well the model predictions match actual values.

13. Feature Engineering

Q27. What is feature scaling?

Feature scaling standardizes data to improve model performance, especially for distance-based algorithms.

Q28. What is one-hot encoding?

One-hot encoding converts categorical variables into binary vectors.

14. Ensemble Learning

Q29. What is bagging?

Bagging reduces variance by training multiple models on different subsets of data.

Q30. What is boosting?

Boosting sequentially trains models, focusing on errors of previous models.

15. Deep Learning Basics

Q31. What is a neural network?

A neural network is a model inspired by the human brain, consisting of layers of interconnected neurons.

Q32. What is activation function?

Activation functions introduce non-linearity into the model.

16. Practical Questions (VERY IMPORTANT)

Q33. How do you handle missing data?

Missing data can be handled by removing rows, imputing values, or using models that handle missing values.

Q34. How do you deal with imbalanced data?

Techniques include oversampling, undersampling, and using appropriate evaluation metrics.

17. Advanced Concepts

Q35. What is ROC-AUC?

ROC-AUC measures the performance of a classification model across thresholds.

Q36. What is confusion matrix?

A confusion matrix shows true positives, false positives, true negatives, and false negatives.

18. Time Series

Q37. What is time series analysis?

Time series analysis deals with data indexed over time to identify trends and patterns.

19. Reinforcement Learning

Q38. What is reinforcement learning?

Reinforcement learning trains agents using rewards and penalties to achieve goals.

20. Real Interview Scenarios

Q39. How would you choose a model?

Model selection depends on data type, problem type, and performance metrics.

Q40. How do you improve model performance?

Performance can be improved through feature engineering, tuning hyperparameters, and using better algorithms.

21. Tricky Questions

Q41. Why is accuracy not always a good metric?

Accuracy can be misleading in imbalanced datasets.

Q42. What is data leakage?

Data leakage occurs when training data contains information that would not be available in real-world predictions.

22. Deployment & Practical ML

Q43. What is model deployment?

Model deployment involves integrating a trained model into a production environment.

Q44. What is A/B testing in ML?

A/B testing compares two models to determine which performs better.

23. Advanced Algorithms

Q45. What is SVM?

Support Vector Machine is a classification algorithm that finds the optimal boundary between classes.

Q46. What is Naive Bayes?

Naive Bayes is a probabilistic classifier based on Bayes' theorem.

24. Big Data ML

Q47. What is scalability in ML?

Scalability refers to handling large datasets efficiently.

25. Final Interview-Level Questions

Q48. Explain end-to-end ML pipeline.

An ML pipeline includes data collection, preprocessing, feature engineering, model training, evaluation, and deployment.

Q49. What are hyperparameters?

Hyperparameters are configuration settings set before training.

Q50. What is difference between parametric and non-parametric models?

Parametric models have fixed parameters, while non-parametric models adapt based on data.

1. Customer Churn Prediction

Q1. A telecom company is facing high customer churn. How would you solve this?

The first step is to clearly define churn, such as customers leaving within a given period. I would collect historical data including customer demographics, usage patterns, billing information, and complaints. Data preprocessing would involve handling missing values, encoding categorical variables, and feature engineering such as tenure or average monthly usage. Since churn is a classification problem, I would start with models like Logistic Regression for interpretability and then try advanced models like Random Forest or Gradient Boosting for better performance. Evaluation would focus on recall and F1-score because identifying churners is more important than overall accuracy. Finally, I would translate results into actionable insights such as identifying high-risk customers and designing targeted retention campaigns.

2. Fraud Detection

Q2. How would you design a credit card fraud detection system?

Fraud detection is a highly imbalanced classification problem where fraudulent transactions are rare. I would begin by preprocessing transaction data, including features like transaction amount, location, time, and frequency. Handling class imbalance is critical, so I would use techniques such as SMOTE, undersampling, or anomaly detection models. Algorithms like Random Forest, XGBoost, or Isolation Forest can be effective. Evaluation should prioritize precision-recall metrics rather than accuracy. Real-time constraints must also be considered, so the model should be optimized for low latency. Business impact involves reducing fraud losses while minimizing false positives to avoid customer inconvenience.

3. Recommendation System

Q3. How would you build a recommendation system for an e-commerce platform?

I would first define the goal, such as increasing user engagement or sales. Then I would collect user behavior data like clicks, purchases, and ratings. The solution could involve collaborative filtering, content-based filtering, or hybrid approaches. Matrix factorization or deep learning models can be used for scalability. Evaluation metrics include precision@k, recall@k, and user engagement metrics. Deployment should ensure real-time recommendations. The business outcome is improved customer experience and higher conversion rates.

4. Sales Forecasting

Q4. A retail company wants to forecast sales. How would you approach this?

This is a time series problem. I would analyze historical sales data along with external factors like seasonality, promotions, and holidays. Feature engineering would include lag variables and rolling averages. Models like ARIMA, Prophet, or LSTM can be used. Evaluation would use metrics like RMSE or MAE. The final model should provide accurate forecasts to optimize inventory and supply chain operations.

5. Customer Segmentation

Q5. How would you segment customers for targeted marketing?

This is an unsupervised learning problem. I would use clustering algorithms such as K-Means or Hierarchical Clustering on features like purchase frequency, spending, and demographics. Feature scaling is important before clustering. The optimal number of clusters can be determined using the elbow method or silhouette score. The segments can then be interpreted for business use, such as identifying high-value customers or discount-sensitive users.

6. Loan Default Prediction

Q6. How would you predict whether a customer will default on a loan?

This is a binary classification problem. I would use historical loan data including income, credit score, and past defaults. Feature engineering and handling missing values are crucial. Logistic Regression is useful for interpretability, while tree-based models can improve performance. Evaluation should focus on recall to minimize false negatives (i.e., approving risky loans). The business goal is reducing financial risk while maintaining customer approval rates.

7. Product Pricing Optimization

Q7. How would you determine optimal pricing for a product?

I would analyze historical sales data and demand elasticity. Regression models can be used to understand how price affects demand. A/B testing can validate pricing strategies. Advanced approaches may involve reinforcement learning for dynamic pricing. The objective is to maximize revenue or profit.

8. Image Classification Use Case

Q8. How would you build a system to classify product images?

This is a computer vision problem. I would preprocess images and use Convolutional Neural Networks (CNNs). Pretrained models like ResNet or VGG can be fine-tuned. Evaluation metrics include accuracy and F1-score. The business application could be automating product categorization.

9. NLP Sentiment Analysis

Q9. How would you analyze customer reviews for sentiment?

I would preprocess text data using tokenization, stop-word removal, and vectorization methods like TF-IDF or word embeddings. Models like Logistic Regression, Naive Bayes, or transformers can be used. Evaluation includes accuracy and F1-score. The outcome helps businesses understand customer feedback.

10. Supply Chain Optimization

Q10. How would you optimize a supply chain using ML?

I would combine demand forecasting with optimization techniques. Predictive models forecast demand, while optimization algorithms minimize costs. Constraints like inventory capacity and delivery time must be considered. The goal is to reduce costs and improve efficiency.

11. Ad Click Prediction

Q11. How would you predict whether a user will click on an ad?

This is a binary classification problem using features like user behavior, demographics, and ad attributes. Models like Logistic Regression or Gradient Boosting are commonly used. Evaluation focuses on AUC-ROC and log-loss. The goal is to improve ad targeting and revenue.

12. Healthcare Prediction

Q12. How would you predict disease risk from patient data?

This involves structured and possibly unstructured data. Feature selection is important. Models must be interpretable, so Logistic Regression or decision trees are preferred. Evaluation includes recall to minimize missed diagnoses. Ethical considerations and data privacy are critical.

13. Handling Imbalanced Data (Case)

Q13. Your dataset is highly imbalanced. What would you do?

I would use resampling techniques like oversampling or undersampling, or apply class-weighted models. Evaluation should rely on precision, recall, and F1-score rather than accuracy.

14. Model Improvement Case

Q14. Your model accuracy is low. What steps would you take?

I would check data quality, perform feature engineering, try different algorithms, tune hyperparameters, and use cross-validation. Ensemble methods can also improve performance.

15. Business Translation (MOST IMPORTANT)

Q15. How do you explain ML results to non-technical stakeholders?

I would focus on business impact rather than technical details. For example, instead of discussing model accuracy, I would explain how the model reduces churn or increases revenue. Visualizations and simple metrics help communicate insights effectively.

PYTHON INTERVIEW QUESTIONS

1. Python Basics

Q1. What is Python?

Python is a high-level, interpreted, and general-purpose programming language known for its simplicity and readability. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming, making it versatile for applications like web development, data science, and automation.

Q2. What are the key features of Python?

Python offers dynamic typing, automatic memory management, a large standard library, cross-platform compatibility, and an easy-to-read syntax. It also has strong community support and is widely used in industries.

Q3. What is the difference between compiled and interpreted languages?

Compiled languages are converted into machine code before execution, whereas interpreted languages like Python execute code line by line at runtime, making debugging easier but sometimes slower.

2. Data Types

Q4. What are built-in data types in Python?

Python provides several built-in data types such as integers, floats, strings, lists, tuples, sets, and dictionaries. Each type serves a different purpose and has unique characteristics.

Q5. What is the difference between list and tuple?

Lists are mutable, meaning their elements can be changed after creation, while tuples are immutable. Lists use square brackets, whereas tuples use parentheses.

Q6. What is a dictionary in Python?

A dictionary is an unordered collection of key-value pairs where each key must be unique. It allows fast retrieval of values based on keys.

3. Control Flow

Q7. What are control statements in Python?

Control statements like if-else, for loops, and while loops control the flow of execution in a program based on conditions or iterations.

4. Functions

Q8. What is a function in Python?

A function is a reusable block of code that performs a specific task. It helps in modular programming and code reusability.

Q9. What are arguments and parameters?

Parameters are variables defined in a function definition, while arguments are the actual values passed when calling the function.

5. Object-Oriented Programming (OOP)

Q10. What is OOP in Python?

Object-Oriented Programming is a paradigm that uses classes and objects to structure code. It supports encapsulation, inheritance, and polymorphism.

Q11. What is inheritance?

Inheritance allows a class to inherit properties and methods from another class, promoting code reuse.

Q12. What is polymorphism?

Polymorphism allows methods to behave differently based on the object calling them.

6. Exception Handling

Q13. What is exception handling?

Exception handling is used to handle runtime errors gracefully using try, except, finally, and else blocks.

7. Memory Management

Q14. How does Python manage memory?

Python uses automatic memory management with a private heap space and garbage collection to free unused memory.

8. Advanced Concepts

Q15. What are decorators?

Decorators are functions that modify the behavior of other functions without changing their code.

Q16. What are generators?

Generators are functions that yield values one at a time using the yield keyword, making them memory efficient.

Q17. What is lambda function?

A lambda function is a small anonymous function defined using the lambda keyword.

9. Data Structures

Q18. What is the difference between list and set?

Lists allow duplicates and maintain order, while sets do not allow duplicates and are unordered.

10. Python Libraries (IMPORTANT)

Q19. What are some popular Python libraries?

Popular libraries include NumPy for numerical computing, Pandas for data manipulation, and Matplotlib for visualization.

11. File Handling

Q20. How does file handling work in Python?

File handling is done using `open()`, `read()`, `write()`, and `close()` functions. Files can be opened in different modes like read, write, or append.

12. Multithreading & Multiprocessing

Q21. What is multithreading?

Multithreading allows multiple threads to run concurrently within a program.

Q22. What is GIL?

The Global Interpreter Lock (GIL) ensures that only one thread executes Python bytecode at a time, which can limit parallelism.

13. Python vs Other Languages

Q23. Why is Python slow compared to C++?

Python is slower because it is interpreted and dynamically typed, whereas C++ is compiled and statically typed.

14. Practical Questions

Q24. How do you reverse a string in Python?

A string can be reversed using slicing, where the string is accessed with a step of -1.

Q25. How do you remove duplicates from a list?

Duplicates can be removed by converting the list into a set and then back to a list.

15. Coding-Oriented Concepts

Q26. What is list comprehension?

List comprehension is a concise way to create lists using a single line of code.

Q27. What is map() and filter()?

map() applies a function to all elements, while filter() selects elements based on a condition.

16. Advanced OOP

Q28. What is method overloading?

Python does not support traditional method overloading, but it can be achieved using default arguments.

Q29. What is method overriding?

Method overriding occurs when a child class provides a specific implementation of a method already defined in the parent class.

17. Iterators

Q30. What is an iterator?

An iterator is an object that allows traversal through elements using **iter()** and **next()** methods.

18. Modules & Packages

Q31. What is a module?

A module is a file containing Python code that can be reused.

Q32. What is a package?

A package is a collection of modules organized in directories.

19. Database Connectivity

Q33. How does Python connect to databases?

Python uses libraries like connectors or ORM tools to interact with databases.

20. Interview-Tricky Questions

Q34. What is mutable vs immutable?

Mutable objects can be changed after creation, while immutable objects cannot.

Q35. What is deep copy vs shallow copy?

A shallow copy copies references, while a deep copy copies all nested objects.

21. Real Interview Scenarios

Q36. How do you optimize Python code?

Optimization involves using efficient data structures, avoiding unnecessary loops, and using built-in functions.

Q37. How do you handle large data in Python?

Large data can be handled using generators, chunk processing, and libraries like Pandas.

22. Advanced Topics

Q38. What is async programming?

Async programming allows non-blocking execution using `async` and `await` keywords.

Q39. What is context manager?

Context managers handle resource management using the `with` statement.

23. Debugging & Testing

Q40. How do you debug Python code?

Debugging can be done using `print` statements, logging, or debugging tools.

24. Final Interview Questions

Q41. What are Python best practices?

Best practices include writing readable code, using meaningful variable names, and following PEP8 standards.

Q42. What is duck typing?

Duck typing means the type of an object is determined by its behavior rather than its class.

25. Scenario-Based Questions

Q43. How would you design a scalable Python application?

Scalability involves modular design, efficient algorithms, and possibly using distributed systems.

Q44. How do you handle errors in production?

Errors are handled using logging, monitoring, and fallback mechanisms.

26. Final Advanced Questions

Q45. What is metaclass?

A metaclass defines the behavior of a class itself.

Q46. What is serialization?

Serialization converts objects into a format for storage or transmission.

27. Important Practical Concepts

Q47. What is virtual environment?

A virtual environment isolates project dependencies.

Q48. What is pip?

pip is the package manager used to install Python libraries.

28. Final Two Questions

Q49. Difference between Python 2 and Python 3?

Python 3 has better Unicode support, improved libraries, and is actively maintained.

Q50. Why is Python popular?

Python is popular due to its simplicity, versatility, and strong ecosystem.

1–10: Basic Coding

Q1. Reverse a string

```
def reverse_string(s):  
    return s[::-1]
```

Q2. Check palindrome

```
def is_palindrome(s):  
    return s == s[::-1]
```

Q3. Factorial

```
def factorial(n):  
    res = 1  
    for i in range(1, n+1):  
        res *= i  
    return res
```

Q4. Fibonacci

```
def fibonacci(n):  
    a, b = 0, 1  
    for _ in range(n):  
        a, b = b, a+b  
    return a
```

Q5. Largest element in array

```
def largest(arr):  
    max_val = arr[0]  
    for num in arr:  
        if num > max_val:  
            max_val = num  
    return max_val
```

Q6. Second largest element

```
def second_largest(arr):  
    first = second = float('-inf')  
    for num in arr:  
        if num > first:  
            second = first  
            first = num  
        elif num > second and num != first:  
            second = num  
    return second
```

Q7. Count vowels

```
def count_vowels(s):  
    count = 0  
    for ch in s.lower():  
        if ch in 'aeiou':
```

```
        count += 1
    return count
```

Q8. Remove duplicates

```
def remove_duplicates(arr):
    return list(set(arr))
```

Q9. Sum of digits

```
def sum_digits(n):
    total = 0
    while n:
        total += n % 10
        n //= 10
    return total
```

Q10. Check prime

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            return False
    return True
```

11–20: Arrays & Strings

Q11. Two sum

```
def two_sum(arr, target):
    d = {}
    for i, num in enumerate(arr):
        if target - num in d:
            return [d[target - num], i]
        d[num] = i
```

Q12. Move zeros to end

```
def move_zeros(arr):
    res = [x for x in arr if x != 0]
    return res + [0]*(len(arr)-len(res))
```

Q13. Find duplicates

```
def find_duplicates(arr):
    seen, dup = set(), set()
    for num in arr:
        if num in seen:
            dup.add(num)
        seen.add(num)
    return list(dup)
```

Q14. Anagram check

```
def is_anagram(s1, s2):
    return sorted(s1) == sorted(s2)
```

Q15. First non-repeating character

```
def first_unique(s):
    for ch in s:
        if s.count(ch) == 1:
            return ch
    return None
```

Q16. Longest common prefix

```
def lcp(arr):
    prefix = arr[0]
    for s in arr[1:]:
        while not s.startswith(prefix):
            prefix = prefix[:-1]
    return prefix
```

Q17. Rotate array

```
def rotate(arr, k):
    k %= len(arr)
    return arr[-k:] + arr[:-k]
```

Q18. Merge two sorted arrays

```
def merge(a, b):
    return sorted(a + b)
```

Q19. Kadane's Algorithm

```
def max_subarray(arr):
```

```

curr = max_sum = arr[0]
for num in arr[1:]:
    curr = max(num, curr+num)
    max_sum = max(max_sum, curr)
return max_sum

```

Q20. Missing number

```

def missing(arr, n):
    return n*(n+1)//2 - sum(arr)

```

21–30: Intermediate

Q21. Majority element

```

def majority(arr):
    count, candidate = 0, None
    for num in arr:
        if count == 0:
            candidate = num
        count += (1 if num == candidate else -1)
    return candidate

```

Q22. Intersection of arrays

```

def intersection(a, b):
    return list(set(a) & set(b))

```

Q23. Union of arrays

```

def union(a, b):
    return list(set(a) | set(b))

```

Q24. Reverse words in string

```

def reverse_words(s):
    return ' '.join(s.split()[::-1])

```

Q25. Valid parentheses

```

def valid_parentheses(s):
    stack = []
    d = {'(': ')', '{': '}', '[': ']'}
    for ch in s:

```

```

if ch in d.values():
    stack.append(ch)
else:
    if not stack or stack.pop() != d[ch]:
        return False
return not stack

```

Q26. Binary search

```

def binary_search(arr, target):
    l, r = 0, len(arr)-1
    while l <= r:
        mid = (l+r)//2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            l = mid+1
        else:
            r = mid-1
    return -1

```

Q27. Power of number

```

def power(x, n):
    res = 1
    for _ in range(n):
        res *= x
    return res

```

Q28. Count frequency

```

def freq(arr):
    d = {}
    for num in arr:
        d[num] = d.get(num, 0) + 1
    return d

```

Q29. Flatten list

```

def flatten(lst):
    res = []
    for i in lst:
        if isinstance(i, list):
            res.extend(flatten(i))
        else:
            res.append(i)

```



```
return res
```

Q30. GCD

```
def gcd(a, b):  
    while b:  
        a, b = b, a % b  
    return a
```

31–40: Advanced Basics

Q31. LCM

```
def lcm(a, b):  
    return (a*b)//gcd(a, b)
```

Q32. Check balanced brackets

```
def balanced(s):  
    stack = []  
    pairs = {'(': ')', '[': ']', '{': '}' }  
    for ch in s:  
        if ch in pairs.values():  
            stack.append(ch)  
        else:  
            if not stack or stack.pop() != pairs[ch]:  
                return False  
    return not stack
```

Q33. Find peak element

```
def peak(arr):  
    for i in range(len(arr)):  
        if (i==0 or arr[i]>=arr[i-1]) and (i==len(arr)-1 or arr[i]>=arr[i+1]):  
            return arr[i]
```

Q34. Product of array except self

```
def product_except_self(arr):  
    res = [1]*len(arr)  
    left = 1  
    for i in range(len(arr)):  
        res[i] = left
```

```

    left *= arr[i]
    right = 1
    for i in range(len(arr)-1, -1, -1):
        res[i] *= right
        right *= arr[i]
    return res

```

Q35. Subarray sum equals k

```

def subarray_sum(arr, k):
    d = {0:1}
    curr = res = 0
    for num in arr:
        curr += num
        res += d.get(curr-k, 0)
        d[curr] = d.get(curr, 0) + 1
    return res

```

Q36. Longest substring without repeating

```

def longest_unique(s):
    d = {}
    l = res = 0
    for r in range(len(s)):
        if s[r] in d:
            l = max(l, d[s[r]]+1)
        d[s[r]] = r
        res = max(res, r-l+1)
    return res

```

Q37. Merge intervals

```

def merge_intervals(intervals):
    intervals.sort()
    res = [intervals[0]]
    for curr in intervals[1:]:
        prev = res[-1]
        if curr[0] <= prev[1]:
            prev[1] = max(prev[1], curr[1])
        else:
            res.append(curr)
    return res

```

Q38. Spiral matrix

```

def spiral(matrix):

```

```

res = []
while matrix:
    res += matrix.pop(0)
    matrix = list(zip(*matrix))[::-1]
return res

```

Q39. Rotate matrix 90

```

def rotate_matrix(mat):
    return list(zip(*mat[::-1]))

```

Q40. Search in rotated array

```

def search_rotated(arr, target):
    l, r = 0, len(arr)-1
    while l <= r:
        mid = (l+r)//2
        if arr[mid] == target:
            return mid
        if arr[l] <= arr[mid]:
            if arr[l] <= target < arr[mid]:
                r = mid-1
            else:
                l = mid+1
        else:
            if arr[mid] < target <= arr[r]:
                l = mid+1
            else:
                r = mid-1
    return -1

```

41–50: Interview Level

Q41. Detect cycle in list (using set)

```

def has_cycle(head):
    seen = set()
    while head:
        if head in seen:
            return True
        seen.add(head)
        head = head.next

```

```
return False
```

Q42. Reverse linked list

```
def reverse_list(head):  
    prev = None  
    while head:  
        nxt = head.next  
        head.next = prev  
        prev = head  
        head = nxt  
    return prev
```

Q43. Find middle of linked list

```
def middle(head):  
    slow = fast = head  
    while fast and fast.next:  
        slow = slow.next  
        fast = fast.next.next  
    return slow
```

Q44. Merge sorted linked lists

```
def merge_lists(l1, l2):  
    dummy = cur = ListNode(0)  
    while l1 and l2:  
        if l1.val < l2.val:  
            cur.next, l1 = l1, l1.next  
        else:  
            cur.next, l2 = l2, l2.next  
        cur = cur.next  
    cur.next = l1 or l2  
    return dummy.next
```

Q45. Climbing stairs

```
def climb(n):  
    if n <= 2:  
        return n  
    a, b = 1, 2  
    for _ in range(3, n+1):  
        a, b = b, a+b  
    return b
```

Q46. Coin change

```
def coin_change(coins, amount):
    dp = [float('inf')] * (amount+1)
    dp[0] = 0
    for coin in coins:
        for i in range(coin, amount+1):
            dp[i] = min(dp[i], dp[i-coin]+1)
    return dp[amount] if dp[amount] != float('inf') else -1
```

Q47. Longest increasing subsequence

```
def lis(arr):
    dp = [1]*len(arr)
    for i in range(len(arr)):
        for j in range(i):
            if arr[i] > arr[j]:
                dp[i] = max(dp[i], dp[j]+1)
    return max(dp)
```

Q48. Word count

```
def word_count(s):
    return len(s.split())
```

Q49. Check anagram (optimized)

```
def anagram(s1, s2):
    if len(s1) != len(s2):
        return False
    d = {}
    for ch in s1:
        d[ch] = d.get(ch, 0) + 1
    for ch in s2:
        if ch not in d or d[ch] == 0:
            return False
        d[ch] -= 1
    return True
```

Q50. Top K frequent elements

```
def top_k(arr, k):
    freq = {}
    for num in arr:
        freq[num] = freq.get(num, 0) + 1
    return sorted(freq, key=freq.get, reverse=True)[:k]
```
